

UNIVERSIDAD TECNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computacion y Diseno Digital

Carrera de Ingenieria de Software (Redisenio)

Examen teorico-practico Unidad IV

Modelo Vista Controlador y Servicios Web:

API REST, Autenticacion JWT y Consumo de APIs Externas

Asignatura: Aplicaciones Web [111] (5to nivel — Paralelo A)

Docente: Dr. Gleiston Ciceron Guerrero Ulloa, Ph.D. gguerrero@uteq.edu.ec

Periodo: 2026-2027 PPA | **Modalidad:** Individual, laboratorio

Duracion: 120 minutos | **Parte A:** 40 % **Parte B:** 60 %

Repositorio base: biblioteca-u4-base (Java 25, Spring Boot 4.1.1)

Apellidos y nombres: _____

Numero de carnet: _____

URL del repositorio: _____

Fecha: _____

1. Instrucciones

El examen tiene dos partes que se rinden de forma consecutiva en la misma sesion de laboratorio. La **Parte A** es teorica, dura 45 minutos y se responde **dentro del repositorio, en el archivo Cuestionario.md de la raiz**, que ya viene creado en el proyecto base con los enunciados y los bloques de respuesta vacios. Se rinde **sin material de consulta y sin conexion a internet**, salvo el acceso al propio repositorio para escribir y hacer *commit*. La **Parte B** es practica, dura 75 minutos y se resuelve en IntelliJ IDEA sobre el repositorio base **biblioteca-u4-base**, con acceso a la documentacion oficial de Spring, a la especificacion OpenAPI y al codigo del propio repositorio, pero **sin asistentes de inteligencia artificial y sin comunicacion con terceros**.

La Parte A se califica sobre 40 puntos y la Parte B sobre 60. La nota sobre diez se obtiene como $\text{Nota}_{10} = (A + B)/10$. Se aprueba con 7,0 o mas, conforme al reglamento academico de la UTEQ.

Criterios de piso (excluyentes): su incumplimiento implica calificacion CERO

1. **Entregable unico en el SGA.** Al cierre del examen debe subir al aula virtual **un unico archivo PDF con caratula** (asignatura, unidad, nombres completos, carnet y fecha) que muestre **claramente, en una sola linea**, la URL del repositorio donde

esta su solución. Si la URL está rota, si el repositorio no es accesible o si no se cumple esta directriz, la calificación es **CERO**.

2. **El proyecto debe compilar.** Si al clonar su repositorio y ejecutar `mvn -B clean package -DskipTests` la compilación falla, la Parte B se califica **CERO**. Verifique la compilación antes de entregar; un proyecto que no compila no se evalúa por partes.
3. **Repositorio público y trazable.** El repositorio abre sin credenciales y su historial muestra *commits* firmados con su nombre y correo institucional, con marca de tiempo dentro de la sesión del examen. Un repositorio subido en un único *commit* posterior al cierre invalida la entrega.
4. **Protocolo de *commits*.** El docente indicará en voz alta los momentos de *commit* durante la sesión. Por cada indicación debe existir un *commit* con marca de tiempo posterior a esa indicación y anterior a la siguiente. Además, **el salto de contenido entre dos *commits* consecutivos debe ser proporcional al tiempo transcurrido**: un *commit* que introduce de golpe un apartado completo que no existía en el anterior, o que supera el umbral de líneas fijado por el docente, **se considera falta grave**, se califica CERO y se reporta al Coordinador de Carrera. Un salto desproporcionado es la firma de código traído de fuera de la sesión.
5. **Trabajo individual.** Código idéntico al de otro estudiante, o generado con asistencia de inteligencia artificial durante la sesión, anula el examen de los implicados y se reporta al Coordinador de Carrera.

Preparación del entorno (haga esto en los primeros cinco minutos)

1. Clone el repositorio base y cree su propio repositorio público a partir de él.
2. `cp .env.example .env` y genere el secreto con `openssl rand -base64 48`.
3. `docker compose up -d postgres redis`
4. `mvn -B clean package -DskipTests` debe terminar en BUILD SUCCESS. Si no, avise de inmediato al docente: es un problema de entorno, no suyo.
5. Haga un primer *commit* de configuración para dejar constancia de su hora de inicio: `git commit -allow-empty -m "checkpoint 0: inicio del examen"` seguido de `git push`.
6. Verifique que `Cuestionario.md` está en la raíz del repositorio: ahí responde la Parte A.

2. Parte A — Teorica (40 puntos, 45 minutos, sin material)

Responda con sus palabras. Se valora la precision tecnica y la justificacion, no la extension: una respuesta correcta de seis lineas vale mas que una pagina imprecisa. Cuando la pregunta pida referirse al proyecto base, hagalo con nombres concretos de clases o de *endpoints*.

Donde y como se responde esta parte

- Las respuestas se escriben en el archivo `Cuestionario.md` de la raiz del repositorio, que ya viene creado con estos mismos enunciados y con un bloque ****Respuesta:**** debajo de cada uno.
- **No borre ni reescriba los enunciados** y no agregue ni quite secciones: el evaluador compara pregunta por pregunta y un archivo reordenado se corrige como incompleto.
- El archivo debe conservar **exactamente** ese nombre y esa ubicacion. Un cuestionario entregado como PDF suelto, como comentario en el codigo o en otro archivo **no se califica**.
- Las respuestas llegan por *commits* sucesivos, segun el protocolo del criterio de piso 4. Un `Cuestionario.md` que aparece completo en un unico *commit* al cierre de la sesion incumple ese criterio.
- Al final del archivo hay una declaracion de honestidad academica que debe marcar y firmar.

A1. Restricciones de REST aplicadas a un caso concreto

(8 puntos)

Fielding define seis restricciones para el estilo arquitectonico REST [1].

- a) Enuncie las **seis** restricciones. (3 puntos)
- b) El proyecto base expone `GET /api/v1/autores` y guarda el estado de la sesion del usuario **solo** en el JWT que el cliente envia en cada peticion. Explique que restriccion concreta se esta cumpliendo con esa decision y **que consecuencia practica** tiene para escalar el sistema a varios servidores detras de un balanceador. (3 puntos)
- c) De las seis restricciones, indique **cual es opcional** y de un ejemplo real de una API que la use. (2 puntos)

A2. Anatomia y ciclo de vida de un JWT

(8 puntos)

- a) Un JWT tiene tres partes separadas por puntos. Nombrelas **en orden** e indique que contiene cada una [2]. (3 puntos)
- b) Un companero afirma: “*como el JWT va firmado, puedo guardar en el payload la contrasena del usuario sin riesgo*”. Explique por que esta equivocado, precisando la diferencia entre **firmar** y **cifrar**. (2 puntos)
- c) El JWT es *stateless* por diseno, lo que genera un problema conocido: no se puede invalidar un token antes de que expire. Describa **dos** estrategias distintas para revocarlo y senale la desventaja de cada una. (3 puntos)

A3. SOAP frente a REST**(8 puntos)**

- a) Complete una tabla comparativa con **seis** criterios entre SOAP y REST. Use al menos: formato del mensaje, contrato de descripción, sobrecarga de serialización, tipado, facilidad de consumo desde un cliente móvil y manejo de errores. (5 puntos)
- b) El Servicio de Rentas Internas del Ecuador expone la autorización de comprobantes electrónicos mediante servicios SOAP. Explique **dos razones técnicas** por las que una institución de ese tipo mantiene SOAP en lugar de migrar a REST. (3 puntos)

A4. Cache-aside sobre un servicio externo**(8 puntos)**

El proyecto base define en `CacheConfig` dos espacios de cache: `libros` con TTL de 2 minutos y `openlibrary` con TTL de 24 horas.

- a) Describa el patrón *cache-aside* en sus cuatro pasos, desde que llega la petición hasta que se responde. (3 puntos)
- b) Justifique técnicamente por qué el TTL de `openlibrary` es doce veces mayor que el de `libros`, y que criterio general debe guiar la elección de un TTL. (3 puntos)
- c) Explique por qué **nunca** debe almacenarse en cache la respuesta de un fallo del servicio externo, y describa que le ocurriría al sistema si se hiciera. (2 puntos)

A5. Diagnostico de codigos de estado y contrato de errores**(8 puntos)**

Para cada escenario indique el **código HTTP correcto** y explique en una línea por qué. Todos los errores del proyecto base salen en formato *Problem Details* conforme a la RFC 9457, que obsoleta a la RFC 7807 [3, 4].

- a) Un cliente pide `GET /api/v1/libros/999999` y ese identificador no existe. (1 punto)
- b) Un cliente envía `POST /api/v1/libros` sin cabecera `Authorization`. (1 punto)
- c) Un usuario autenticado con rol `LECTOR` envía `POST /api/v1/libros`. (1 punto)
- d) Un cliente envía `POST /api/v1/libros` con el campo `titulo` vacío. (1 punto)
- e) Un cliente intenta prestar un libro a un socio que ya tiene tres préstamos activos. (1 punto)
- f) La API de Open Library no responde dentro del *timeout* configurado. (1 punto)
- g) Explique por qué devolver `200 OK` con un cuerpo `{"success": false}` es un error de diseño, y que restricción de REST se incumple al hacerlo. (2 puntos)

3. Parte B — Practica (60 puntos, 75 minutos, en IntelliJ IDEA)

Contexto del problema

La Biblioteca Municipal de Quevedo esta migrando su catalogo al sistema del proyecto base. Las capas de dominio, persistencia y logica de negocio **ya estan implementadas y probadas**: `LibroService`, `LibroMapper`, `LibroSpecs`, `GlobalExceptionHandler`, `ApiResponse` y `CacheConfig` funcionan y no deben modificarse. Su tarea es **exponer y proteger** esa funcionalidad como servicio web, que es exactamente el objeto de la Unidad IV.

El controlador `AutorController` esta implementado y comentado paso a paso: es el patron de referencia que debe replicar. Copiar su estructura no solo esta permitido, es lo esperado.

Entregables de la Parte B

1. Repositorio publico con sus *commits* fechados dentro de la sesion.
2. El proyecto compila con `mvn -B clean package -DskipTests`.
3. PDF con caratula y la URL del repositorio en una sola linea, subido al SGA.

B1. Listado de libros con filtros, paginacion y envoltorio

(18 puntos)

Implemente en `LibroController` el *endpoint* `GET /api/v1/libros`.

- Acepta los parametros opcionales `titulo`, `categoriaId` y `anioDesde`, y los delega a `LibroService.buscar(...)`, que ya existe. (5 puntos)
- Acepta paginacion con `Pageable` y tamano por defecto de 20. (3 puntos)
- Devuelve 200 con el envoltorio `{success, data, message, errors, meta}`, donde `data` es la lista de `LibroResponse` y `meta` son los metadatos de `PageMeta`. (7 puntos)
- Las URI respetan las convenciones REST: sustantivo en plural, versionado `/v1`, sin verbos. (3 puntos)

Verificacion: `curl "http://localhost:8080/api/v1/libros?titulo=clean&size=5"` devuelve 200 y el campo `meta.totalElements` es un numero.

B2. Alta de libros protegida por rol

(14 puntos)

Implemente `POST /api/v1/libros`.

- Recibe un `LibroRequest` validado con `@Valid`. Un cuerpo invalido debe producir 400 en formato *Problem Details* con el arreglo `errors` poblado. **No escriba usted ese manejo**: ya existe en `GlobalExceptionHandler`; lo que se evalua es que no lo interfiera. (4 puntos)
- Devuelve **201 Created** con cabecera `Location` apuntando al recurso creado. (5 puntos)
- Solo el rol `ADMIN` puede invocarlo. Un usuario autenticado con otro rol recibe 403; una peticion sin token recibe 401. (5 puntos)

Nota: para que la proteccion por rol funcione debe completar tambien el TODO-U4-2 minimo: emision del JWT en `AuthController`, validacion en `JwtAuthenticationFilter` y la cadena de `SecurityConfig`. Si no alcanza a terminar la seguridad, implemente igual la anotacion `@PreAuthorize`: se otorgan los puntos de la declaracion aunque el filtro quede incompleto.

B3. Consumo de la API externa con cache y manejo de fallos (16 puntos)

Implemente `OpenLibraryClient.consultarPorIsbn(String isbn)` y expóngalo en `GET /api/v1/libros/{id}/enriquecido`, que devuelve un `LibroEnriquecidoResponse` combinando el libro local con los metadatos externos.

- El consumo se hace **desde el servidor** con el *bean* `RestClient` ya configurado, que trae los *timeout* puestos. (3 puntos)
- Aplica *cache-aside* sobre el espacio `CacheConfig.CACHE_OPENLIBRARY`. (5 puntos)
- Trata de forma **diferenciada** los fallos: un 404 del proveedor devuelve los campos externos en `null` y el *endpoint* responde 200 con el libro local; un *5xx*, un *4xx* distinto de 404 o un *timeout* lanzan `ServicioExternoException`, que el manejador global convierte en 502. (6 puntos)
- **No cachea los fallos.** (2 puntos)

Verificación: tras dos llamadas seguidas al mismo libro, `docker compose exec redis redis-cli KEYS "openlibrary*"` muestra la clave almacenada. Incluya esa captura en su PDF.

B4. Pruebas de integracion (12 puntos)

Escriba en `LibroControllerIT` **tres** pruebas de integracion que hereden de `BaseIntegracionTest`, cada una con su `@DisplayName` descriptivo:

1. `GET /api/v1/libros` responde 200 y el cuerpo trae las cinco claves del envoltorio, con `meta.page` y `meta.size` correctos. (4 puntos)
2. `GET /api/v1/libros/999999` responde 404 y el cuerpo trae `title`, `status` y `detail` del formato *Problem Details*. (4 puntos)
3. `POST /api/v1/libros` con `titulo` vacío responde 400 y el arreglo `errors` no está vacío. (4 puntos)

Las tres deben pasar con `mvn -B test`. Una prueba que no ejecuta no puntúa, y una prueba con aserciones triviales (por ejemplo `assertTrue(true)`) puntúa cero.

Recomendacion de administracion del tiempo

Un reparto razonable de los 75 minutos es: 20 para B1, 15 para B2, 25 para B3 y 15 para B4.

Haga *commit* al terminar cada apartado, además de los que indique el docente: un apartado terminado y *commiteado* puntúa completo aunque el siguiente quede a medias, mientras que un único *commit* final que no compila hace perder toda la Parte B por el criterio de piso 2. Conviene *commitear* incluso código que aun no compila del todo: un historial de diez o doce *commits* pequeños es evidencia a su favor, mientras que dos *commits* enormes son evidencia en su contra aunque el código sea suyo.

4. Planilla de calificacion

Apartado	Puntos	Obtenido	Evidencia verificada
A1. Restricciones de REST	8		Cuestionario.md
A2. Anatomia del JWT	8		Cuestionario.md
A3. SOAP frente a REST	8		Cuestionario.md
A4. Cache-aside y TTL	8		Cuestionario.md
A5. Codigos de estado	8		Cuestionario.md
Subtotal Parte A	40		
B1. Listado con filtros y meta	18		curl + lectura de codigo
B2. Alta protegida por rol	14		curl con y sin token
B3. API externa con cache	16		redis-cli KEYS + codigo
B4. Pruebas de integracion	12		mvn -B test en verde
Subtotal Parte B	60		
TOTAL	100		$\text{Nota}_{10} = \text{Total}/10$

Los criterios de piso se verifican **antes** de aplicar esta planilla. Si alguno falla, no se emite calificacion por apartado.

Referencias

- [1] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. Ph.d. dissertation, University of California, Irvine, 2000.
- [2] M. B. Jones, J. Bradley, and N. Sakimura, “JSON web token (JWT),” Request for Comments RFC 7519, Internet Engineering Task Force (IETF), May 2015.
- [3] M. Nottingham, E. Wilde, and S. Dalal, “Problem details for HTTP APIs,” Request for Comments RFC 9457, Internet Engineering Task Force (IETF), July 2023. Obsoletes RFC 7807.
- [4] R. T. Fielding, M. Nottingham, and J. Reschke, “HTTP semantics,” Request for Comments RFC 9110, Internet Engineering Task Force (IETF), June 2022. STD 97.